



使用JDBC修改数据

北京理工大学计算机学院
金旭亮

概述



在数据库应用程序中，数据的修改，主要分为“新增”、“删除”和“修改”三类，这三种类型的操作，都对应着相应的SQL命令：

数据操作	对应的SQL命令
新增	<code>insert into 表名...</code>
删除	<code>delete from 表名</code>
修改	<code>update 表名 set ...</code>

使用JDBC实现数据的修改，其实就是拼接出上述三种类型的SQL命令，然后发给数据库执行.....

数据的插入

数据库使用insert SQL命令插入数据，JDBC则调用Statement对象的executeUpdate()方法完成数据的插入任务。

//测试数据的插入

```
private static void testInsert() {  
    try (Connection connection = DriverManager.getConnection(dbUrl);  
        Statement statement = connection.createStatement()) {  
        int ranValue = new Random().nextInt(bound: 100);  
        //生成SQL命令  
        String clientName = "client" + ranValue;  
        String address = "address of client " + ranValue;  
        //插入数据时，可以不指定自增字段的值，而由数据库自己生成  
        String sql = "insert into OrderClient(ClientName,Address)" +  
            "values('" + clientName + "','" + address + "');" +  
        int rowAffected = statement.executeUpdate(sql);  
        if (rowAffected > 0) {  
            System.out.println("数据已经成功插入");  
        } else {  
            System.out.println("插入操作未成功");  
        }  
    } catch (SQLException e) {  
        System.err.println(e.getMessage());  
    }  
}
```

数据的更新

更新数据与插入数据功能的实现是类似的，只是它使用的SQL命令是update而不是insert。

```
//测试更新数据
private static void testUpdate() {
    try (Connection connection = DriverManager.getConnection(dbUrl);
        Statement statement = connection.createStatement()
    ) {
        // 更新数据的SQL命令
        String sql = "update OrderClient set ClientName='张三' " +
            " where ClientName='李四'";
        int rowAffected = statement.executeUpdate(sql);
        if (rowAffected > 0) {
            System.out.println("数据已经成功更新:" + rowAffected);
        } else {
            System.out.println("更新操作未成功");
        }
    } catch (SQLException e) {
        System.err.println(e.getMessage());
    }
}
```

删除数据

```
//测试删除数据
private static void testDelete() {
    try (Connection connection = DriverManager.getConnection(dbUrl);
        Statement statement = connection.createStatement()) {
        //删除数据
        String sql = "delete from OrderClient where ClientName like 'client%'";
        int rowAffected = statement.executeUpdate(sql);
        if (rowAffected > 0) {
            System.out.println("数据已经成功删除:" + rowAffected);
        } else {
            System.out.println("没有删除任何一条数据");
        }
    } catch (SQLException e) {
        System.err.println(e.getMessage());
    }
}
```



Note: 当数据库中各表存在关联时，通常应该先删除子表数据，再删除父表记录。也可以在数据库设置为“**级联删除**”，但要小心操作。

可更新的ResultSet-1



除了使用SQL命令直接更新数据库，JDBC还提供另一种通过修改ResultSet字段的方式更新数据库。

(1) 在创建statement对象时指定相应的参数，以得到可更新的ResultSet:

```
String sql = "select * from admin where Id=?";  
PreparedStatement stmt = conn.prepareStatement( sql,  
        ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE);
```

(2) 设置参数，执行select命令提取数据，得到一个可更新的ResultSet对象:

```
stmt.setInt(1, 100);  
ResultSet rs = stmt.executeQuery();
```

可更新的ResultSet-2



现在，更新ResultSet中的字段，最后，调用updateRow()方法更新数据库：

```
if(rs.next()){  
    rs.updateString("userName", bean.getUserName());  
    rs.updateString("password", bean.getPassword());  
    rs.updateRow();  
}
```

```

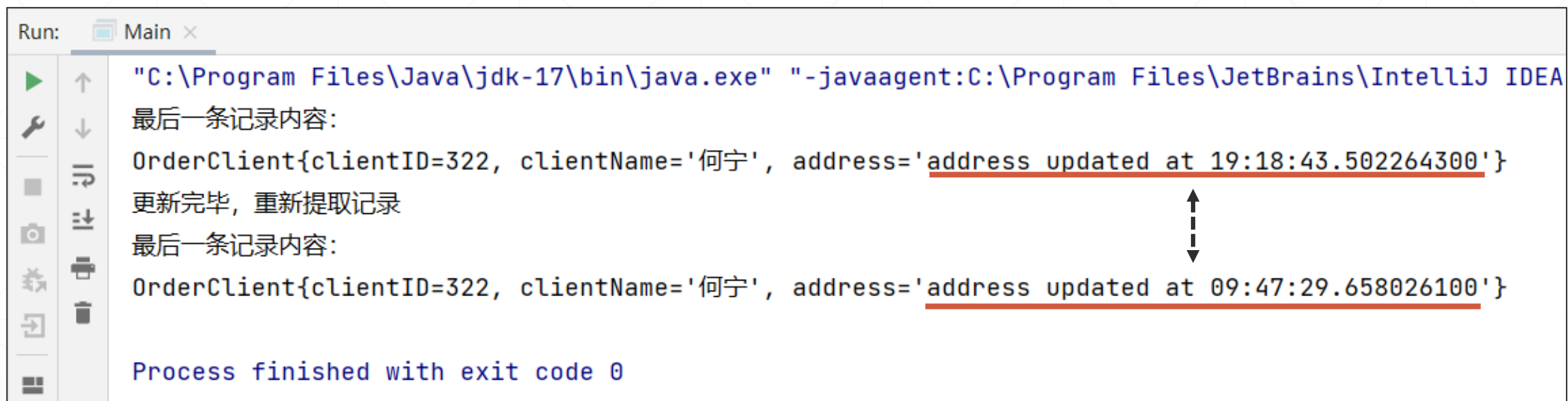
private static void testUpdateResultSet() {
    try (Connection connection =
        DriverManager.getConnection(MsSqlServer, userName, password);
        Statement statement = connection.createStatement(
            ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_UPDATABLE);) {
        String sql = "select * from OrderClient";
        var rs : ResultSet = statement.executeQuery(sql);
        rs.last();
        System.out.println("最后一条记录内容: ");
        System.out.println(getClientFromResultSet(rs));
        rs.updateString( columnLabel: "address",
            x: "address updated at " + LocalTime.now());
        rs.updateRow();
        System.out.println("更新完毕, 重新提取记录");
        var rs2 : ResultSet = statement.executeQuery(sql);
        rs2.last();
        System.out.println("最后一条记录内容: ");
        System.out.println(getClientFromResultSet(rs2));
    } catch (SQLException e) {
        System.err.println(e.getMessage());
    }
}

```

先更新字段，再更新行。

SQLite只支持TYPE_FORWARD_ONLY类型的cursor
因此，此示例代码必须使用SQL Server这样的数据库。

运行截图



```
Run: Main x
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA
最后一条记录内容:
OrderClient{clientId=322, clientName='何宁', address='address updated at 19:18:43.502264300'}
更新完毕, 重新提取记录
最后一条记录内容:
OrderClient{clientId=322, clientName='何宁', address='address updated at 09:47:29.658026100'}

Process finished with exit code 0
```

通过ResultSet更新数据库需要注意：



有效的组合方式：

```
ResultSet.TYPE_SCROLL_SENSITIVE,  
ResultSet.CONCUR_UPDATABLE
```



无效的组合方式：

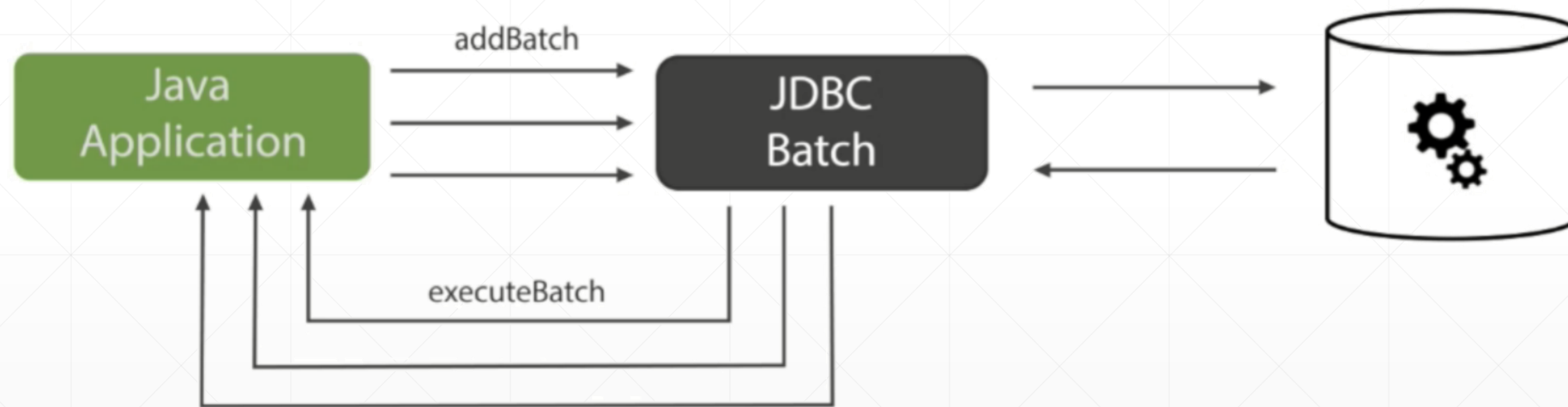
```
ResultSet.TYPE_SCROLL_INSENSITIVE,  
ResultSet.CONCUR_UPDATABLE
```

否则，JDBC会报错：**不支持此游标类型/并发组合。**

一次性地执行多条语句



如果应用程序需要执行多个INSERT语句，将多条数据插入数据库，这时，JDBC可以通过**批量更新**特性来提高程序的运行性能。



在使用批量更新特性时，多条命令将作为一个整体，集中提交。

批量更新的实现



使用批量更新的方法定义在Statement接口中，通过方法名即可“望名知义”，知道它的职责是什么.....

- public void **addBatch**(String sql) throws SQLException
- public int[] **executeBatch**() throws SQLException
- public void **clearBatch**() throws SQLException



批量更新中的命令可以是INSERT、UPDATE和DELETE操作命令，也可以是数据定义命令，但是，不能将SELECT命令添加给批量更新操作，因为执行SELECT命令将会返回一个ResultSet。



使用时，批量更新经常会与事务处理一起应用。

批量更新示例



//测试数据的插入

```
private static void testInsertBatch() {  
    try (Connection connection = DriverManager.getConnection(dbUrl);  
        Statement statement = connection.createStatement()) {  
        statement.addBatch(sql: "insert into OrderClient(ClientName,Address) values('张三','北京')");  
        statement.addBatch(sql: "insert into OrderClient(ClientName,Address) values('李四','上海')");  
        statement.addBatch(sql: "insert into OrderClient(ClientName,Address) values('王五','广州')");  
        statement.addBatch(sql: "insert into OrderClient(ClientName,Address) values('赵六','深圳')");  
  
        var result :int[] = statement.executeBatch();  
        var arrString :String = Arrays.stream(result).mapToObj(String::valueOf)  
            .collect(Collectors.joining(delimiter: ","));  
        System.out.println(arrString);  
    } catch (SQLException e) {  
        System.err.println(e.getMessage());  
    }  
}
```

"批量更新"示例运行截图

```
Run: Main x
"C:\Program Files\Java\jdk-17\bin\java.exe"
1,1,1,1
Process finished with exit code 0
```

main.OrderClient			
4 rows			
	ClientID	ClientName	Address
1	32	张三	北京
2	33	李四	上海
3	34	王五	广州
4	35	赵六	深圳

执行了四条insert命令，得到一个包容四个整数的int[]数组，其中的值均为1，查看数据库，可以看到记录确实插入到了数据库中。

下一讲



事务处理